

SIEM Detection Lab Report

Day 8 — Unauthenticated VNC Access, Credential Exposure & Systemd Service Privilege Escalation

Student Name:	Randy Escototo
Batch:	Batch7
Target Host:	10.10.1.58 (ubuntutarget)
Attacker Host:	10.10.1.10 (Kali)
Vulnerabilities:	CWE-200 (Sensitive Info Exposure) / CWE-256 (Plaintext Credential Storage)
Privesc Method:	Misconfigured sudo → malicious systemd service → SUID bash copy
Date:	August 4–5, 2026

1. Exploitation Narrative

The following describes the full attack chain executed against the target (10.10.1.58): from host discovery through unauthenticated VNC access, NATO phonetic credential decoding, SSH login, malicious systemd service creation, and privilege escalation to root via a SUID bash copy.

Step 1 — Host Discovery

Command: `ping -c 4 10.10.1.58`

Why: Confirmed the target was alive on the internal network. All 4 ICMP packets returned successfully, confirming a live host before further enumeration.

Step 2 — Initial Port Scan

Command: `nmap -sT -p 22,80,443 10.10.1.58`

Why: Checked common web and SSH ports first. Only port 22 (SSH) was open; HTTP and HTTPS were closed. No web attack surface, so a broader scan was needed.

Step 3 — Full Service/Version/OS Scan

Command: `nmap -sV -sC -O 10.10.1.58`

Why: A full service scan revealed two open ports: 22/tcp (OpenSSH 10.2p1) and critically 5901/tcp (VNC protocol 3.8). Nmap's VNC script reported: **Security types: None (1) — Server does not require authentication.** This is the primary vulnerability: a VNC server exposed with no password protection.

Step 4 — VNC Connection (Unauthenticated)

Command: `vncviewer 10.10.1.58:5901`

Why: Connected to the VNC server on port 5901 using TightVNC viewer. No password was required — the server accepted the connection immediately. The remote desktop of the target machine (ubuntutarget:1) became visible, constituting unauthorized access to the target's graphical session.

Step 5 — Credential Discovery via VNC Screen

Observed: The VNC screen displayed a terminal with the following message:

```
Remember, before running the audit script, always verbally confirm the
daily checkphrase with your shift partner. I always say it backwards to
avoid tripping the voice filters (don't ask...).
```

Phrase:

```
romeo Thre3 delta november india mike Thre3 ROMEO CHARLIE NOVEMBER VICTOR delta
lima Zer0
```

Why: The phrase is a NATO phonetic alphabet encoding of a password, stated in reverse (as noted by the message: "I always say it backwards"). Reading the NATO letters from right to left:

Zer0→**0**, lima→**l**, delta→**d**, VICTOR→**V**, NOVEMBER→**N**, CHARLIE→**C**, ROMEO→**R**, Thre3→**3**, mike→**m**,
india→**i**, november→**n**, delta→**d**, Thre3→**3**, romeo→**r**

Decoded password: **0ldVNCR3mind3r**

Step 6 — SSH Login with Discovered Credential

Command: `ssh student@10.10.1.58 (password: 0ldVNCR3mind3r)`

Why: Used the password decoded from the VNC screen to authenticate over SSH, gaining an interactive low-privilege shell as the `student` user on Ubuntu 26.04 LTS.

Step 7 — Creating a Malicious Systemd Service

Command: `sudo systemctl edit --force --full test.service`

Why: The `student` account had `sudo` access to `systemctl`. A malicious service unit file was created with the following content:

```
[Service]
ExecStart=/bin/bash -c 'cp /bin/bash /var/tmp/rootbash && chmod 4755 /var/tmp/rootbash'
[Install]
WantedBy=multi-user.target
```

This service, when started as `root`, copies `/bin/bash` to `/var/tmp/rootbash` and sets the SUID bit (4755), allowing any user to execute it with root privileges. Multiple attempts were needed due to an initially missing `[Unit]` section causing unit file validation errors.

Step 8 — Reloading and Restarting the Service

Commands:

```
sudo systemctl daemon-reload
sudo systemctl restart test.service
```

Why: After correcting the service file, the daemon was reloaded to pick up the new unit file, then the service was restarted. Since the service runs as `root` via `systemd`, the `ExecStart` command executed with root privileges, creating the SUID rootbash binary in `/var/tmp/`.

Step 9 — Executing the SUID Bash Copy

Command: `/var/tmp/rootbash -p`

Why: The `-p` flag tells bash to preserve the effective user ID (EUID) rather than dropping it to the real UID. Since the binary has the SUID bit set and is owned by `root`, executing it with `-p` spawns a root shell without requiring a password.

Step 10 — Root Confirmation

Command: `whoami`

Why: Confirmed full privilege escalation. The command returned `root` inside the `rootbash-5.3#` prompt, verifying complete administrative control of the target machine.

2. Root Proof

2a. VNC Screen — Credential Exposure

The screenshot below shows the TightVNC desktop session obtained without any authentication. The screen displays the NATO phonetic phrase encoding the plaintext password 0ldVNCR3mind3r in reverse order.

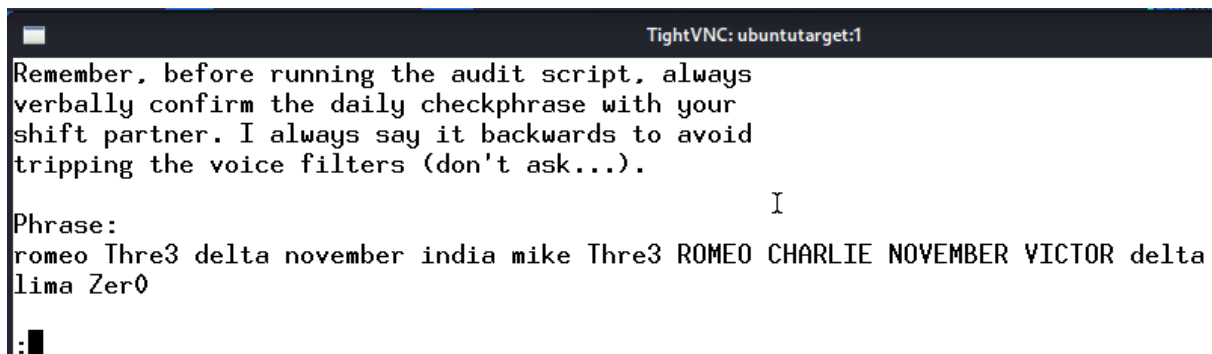


Figure 1: Unauthenticated VNC session showing the NATO phonetic phrase containing the encoded password (reversed: 0ldVNCR3mind3r).

2b. Privilege Escalation & Root Confirmation

The screenshot below shows the terminal session on the target machine. After the malicious systemd service successfully ran and created the SUID rootbash binary, executing `/var/tmp/rootbash -p` spawned a root shell. `whoami` returns `root` inside the `rootbash-5.3#` prompt.

```
student@ubuntutarget: ~  
Session Actions Edit View Help  
student@ubuntutarget:~$ sudo systemctl restart test.service  
Failed to restart test.service: Unit test.service has a bad unit file setting  
See system logs and 'systemctl status test.service' for details.  
student@ubuntutarget:~$ sudo systemctl edit --force --full test.service  
Successfully installed edited file '/etc/systemd/system/test.service'.  
student@ubuntutarget:~$ sudo systemctl daemon-reload  
student@ubuntutarget:~$ sudo systemctl daemon-reload  
student@ubuntutarget:~$ sudo systemctl restart test.service  
Failed to restart test.service: Unit test.service has a bad unit file setting  
See system logs and 'systemctl status test.service' for details.  
student@ubuntutarget:~$ sudo systemctl edit --force --full test.service  
Successfully installed edited file '/etc/systemd/system/test.service'.  
student@ubuntutarget:~$ ^C  
student@ubuntutarget:~$ sudo systemctl daemon-reload  
student@ubuntutarget:~$ sudo systemctl restart test.service  
student@ubuntutarget:~$ /var/tmp/rootbash -p  
rootbash-5.3# whoami  
root  
rootbash-5.3#  
::1          ff02::2          ip6-localnet    ubuntu_target  
fe00::0      ip6-allnodes    ip6-loopback  
ff00::0      ip6-allrouters  ip6-mcastprefix  
ff02::1      ip6-localhost   localhost  
rootbash-5.3# whoami  
root  
rootbash-5.3#
```

Figure 2: Terminal showing malicious systemd service restart, SUID rootbash execution, and whoami returning root.

3. Detection Evidence (SIEM)

The Kibana Discover view below, filtered to `source.ip : "10.10.1.10"` (Kali), captures **1,245 documents** over approximately 2.5 hours (03:44 – 06:44 on August 5, 2026). Two distinct activity spikes are visible in the histogram: a large burst at ~05:25–05:30 corresponding to the VNC connection and credential discovery, and a second cluster at ~05:45–05:55 corresponding to the SSH login and systemd service exploitation phase. Key event types include `rfb` (VNC/Remote Frame Buffer protocol), `ssh` events (login and session activity), `flow` events (port scan traffic), and `alert` events where Suricata rules fired on suspicious activity.

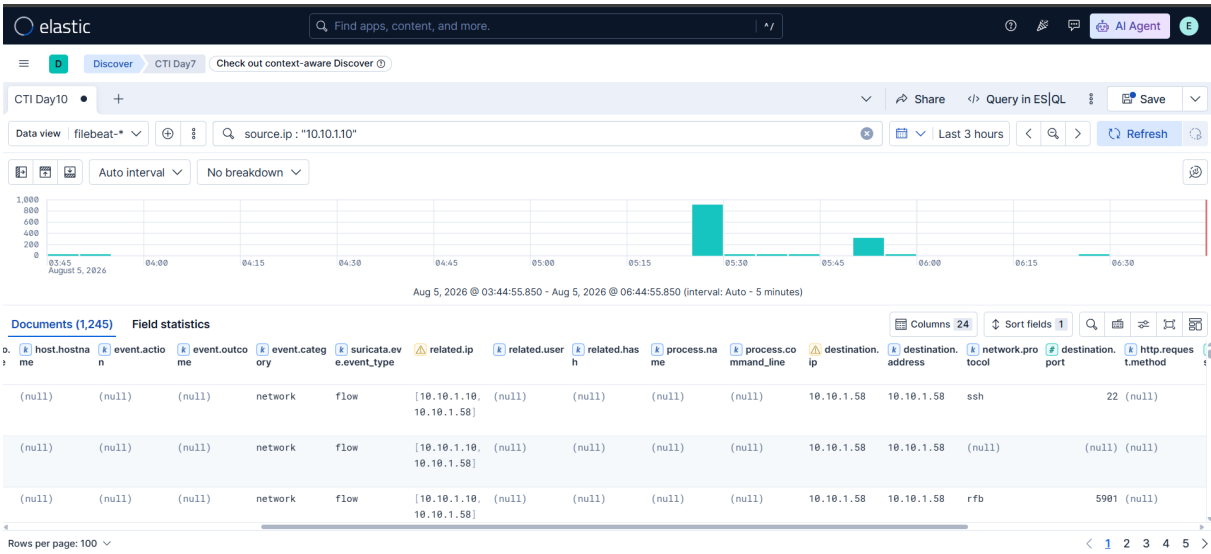


Figure 3: Kibana Discover view (1,245 events, Aug 5 03:44–06:44) filtered to attacker IP 10.10.1.10 against target 10.10.1.58. Visible event types include `flow` (port scan), `rfb` (VNC), `ssh` (login/session), and `alert` (Suricata).

Event Type	Count	Protocol	Significance
flow	1,193	ssh / rfb / rdp / tls	Port scans, VNC and SSH session flows
ssh	19	ssh	Multiple SSH login attempts + successful session
rfb	13	rfb (VNC port 5901)	Unauthenticated VNC connection (key detection signal)
alert	13	rdp / —	Suricata rules firing on suspicious remote access activity
tls	4	tls	TLS handshake events during session
anomaly	3	—	Protocol anomaly events during enumeration

4. Why This Is Dangerous & How SIEM Can Alert on It

The Vulnerabilities

- **Unauthenticated VNC (CWE-200 — Exposure of Sensitive Information):** The VNC server on port 5901 was configured with security type None, meaning any attacker with network access could connect to the graphical desktop session without providing any credentials. This directly exposed everything visible on the target's screen, including sensitive information left in open terminals.
- **Password stored/displayed in plaintext on screen (CWE-256 — Plaintext Storage of Password):** The password 0ldVNCr3mind3r was displayed in a terminal window as a NATO phonetic phrase. While obfuscated by encoding and reversal, it was effectively a plaintext credential visible to anyone who could see the screen — and the unauthenticated VNC meant anyone on the network could see the screen.
- **Overly permissive sudo on systemctl:** The student account had unrestricted sudo access to `systemctl`, allowing creation and execution of arbitrary system services as root. A malicious service that creates a SUID bash copy is a well-known privilege escalation technique that requires no exploits — only write access to systemd unit files.

Why It Is Dangerous

The attack chain is entirely credential-free in its first phase: an attacker with only network access can see the target's desktop without any login. The credential visible on that screen then provides SSH access, and a single misconfigured sudo rule escalates to root. No CVEs or exploit code were required. The combination of CWE-200 and CWE-256 means sensitive information is both stored insecurely and exposed to unauthenticated parties simultaneously. The 1,245 SIEM events generated across 2.5 hours demonstrate the sustained and detectable nature of the attack.

How SIEM Could Alert on This Activity

- **Unauthenticated VNC connection detection:** The 13 `rfb` (Remote Frame Buffer) events in the SIEM directly capture the VNC connection. A Kibana alert on any `rfb` event from an external or untrusted source IP would immediately flag unauthorized VNC access, especially from IPs with no prior legitimate VNC history.
- **VNC with no-auth security type:** Suricata has built-in RFB/VNC protocol analysis. A rule detecting VNC sessions where the negotiated security type is None (security type 1) would directly flag the misconfiguration as soon as any client connects.
- **SSH login from same IP as VNC:** Correlating an `rfb` session followed shortly by a successful `ssh` login from the same source IP is a strong signal of credential harvesting via VNC. The SIEM shows exactly this pattern in the two activity clusters visible in the histogram.
- **Systemd service creation/modification via sudo:** Enabling `auditd` on the target and shipping logs via Filebeat would capture `sudo systemctl edit --force --full test.service` and `sudo systemctl restart test.service` being run by the student account. Alerting on sudo invocations of `systemctl edit` or `systemctl restart` for non-standard service names would flag this escalation attempt.

- **SUID binary creation in /var/tmp:** File integrity monitoring (FIM) rules watching /var/tmp/ for newly created SUID binaries (files with mode 4755 or similar) would immediately flag rootbash being created as a highly suspicious event indicating privilege escalation preparation.
- **Suricata alerts already firing:** The 13 Suricata alert events in the SIEM evidence indicate existing IDS rules partially detected the attack. Configuring automated Kibana alerting on these Suricata rule firings would notify a SOC analyst in real time rather than requiring manual discovery.

Remediation

- **Require VNC authentication:** Configure the VNC server to require a strong password (security type 2 or higher). Better: disable VNC entirely and use SSH with key authentication for remote access.
- **Never display credentials on screen:** Remove the practice of storing or displaying passwords in any form (including encoded/phonetic) in terminal windows or shared desktop sessions.
- **Restrict systemctl sudo access:** Remove systemctl from the student account's sudo permissions entirely. If service management is required, use a dedicated service account with tightly scoped permissions rather than a broad systemctl grant.
- **Network-level controls:** VNC port (5901) should not be exposed on the internal network without strict firewall rules limiting access to authorized management IPs only.